22

エージェント型コーディング eval におけるインフラノイズを定量化する

— Quantifying infrastructure noise in agentic coding evals —

原題	Quantifying infrastructure noise in agentic coding evals
著者	Anthropic Engineering Team
原文	https://www.anthropic.com/engineering/infrastructure-noise
翻訳	Claude（機械翻訳／Anthropic）
編集	2026-04-09

エージェント型コーディング eval におけるインフラノイズを定量化する

SWE-bench や Terminal-Bench のようなエージェント型コーディングベンチマークは、最先端モデルのソフトウェアエンジニアリング能力を比較するのに一般的に使われ、リーダーボードの上位はしばしばわずか数ポイントで分かれます。これらのスコアはモデルの相対能力の精密な測定として扱われ、どのモデルをデプロイするかの決定をますます左右しています。しかし、インフラ設定だけで、その余白を超える差を生み得ることを私たちは発見しました。内部実験では、Terminal-Bench 2.0 で最もリソースが豊富なセットアップと最も乏しいセットアップとの差は 6 パーセントポイントでした（$p < 0.01$）。

静的ベンチマークはモデルの出力を直接採点します——ランタイム環境は結果に影響しません。エージェント型コーディング eval は異なります——モデルには、プログラムを書き、テストを走らせ、依存関係をインストールし、複数ターンで反復する完全な環境が与えられます。ランタイムはもはや受動的なコンテナではなく、問題解決プロセスの不可欠なコンポーネントです。異なるリソース予算と時間制限を持つ 2 つのエージェントは、同じテストを受けていません。

eval 開発者はこれを考慮に入れ始めています。たとえば Terminal-Bench 2.0 は、最新 2.0 リリースでタスクごとの推奨 CPU と RAM を指定します。しかし、リソースを指定することと一貫して強制することは同じではありません。さらに、強制手法がベンチマークが最終的に何を測っているかを変え得ることを発見しました。

どうしてこうなったか

Terminal-Bench 2.0 を Google Kubernetes Engine クラスタで走らせています。セットアップをキャリブレーション中、スコアがベンチマーク公式リーダーボードと合わず、インフラエラー率が驚くほど高いことに気づきました——6% ものタスクが pod エラーで失敗しており、その大半はモデルのタスク解決能力とは無関係でした。

スコアの不一致は強制に行き着きました。私たちの Kubernetes 実装は、タスクごとのリソース仕様を最小保証と厳格な上限の両方として扱いました——各コンテナは指定されたリソースを保証されますが、それを超えた瞬間に殺されます。コンテナランタイムはリソースを 2 つの別々のパラメータで強制します——保証された配分（前もって予約されるリソース）と、コンテナが殺される厳格な上限です。これらを同じ値に設定すると、一時的なスパイクの余裕がゼロになります——一瞬のメモリ変動が、それ以外では成功していたはずのコンテナを OOM-kill できます。これを考慮に入れるため、Terminal-Bench のリーダーボードは異なるサンドボックスプロバイダを使います——その実装はより寛容で、インフラ安定性のために一時的な過剰割り当てを許容しコンテナを終了させません。

この発見はより大きな疑問を提起しました——リソース設定は evaluation スコアにどれほど影響するのか?

スキャフォールドの効果を定量化するため、Terminal-Bench 2.0 を 6 つのリソース構成で走らせました——タスクごとの仕様の厳格強制(1x、最小・上限の両方として)から完全にキャップなしまで。それ以外はすべて一定です——同じ Claude モデル、同じハーネス、同じタスクセット。

実験では、成功率はリソースの余裕と共に上がりました。これは主にインフラエラー率が各ステップで単調に下がることに駆動され、厳格強制の 5.8% から、キャップなしの 0.5% まで。厳格強制から 3x ヘッドルームへの低下(5.8% から 2.1%)は $p < 0.001$ で有意でした。余裕が多ければ、配分を超えて殺されるコンテナが少なくなります。

1x から 3x までは、成功スコアはノイズの余白内で変動します($p=0.40$)。1x でクラッシュしていたタスクのほとんどは、どのみち失敗していたはずです——これはデータから観察できました。エージェントは探索し、リソースの壁に当たり、プリエンプトされますが、元々正解への道にはいませんでした。

しかし、およそ 3x 前後から、この傾向が変わります——成功率がインフラエラーの減少より速く登り始めます。

3x からキャップなしの間で、インフラエラーはさらに 1.6 パーセントポイント下がる一方、成功はほぼ 4 パーセントポイント跳ね上がります。余分なリソースは、寛大な配分でしか動かないアプローチ——大きな依存関係の取り込み、高価なサブプロセスの spawn、メモリ集約的なテストスイートの実行——をエージェントに試させます。キャップなしリソースでは、1x に対する総上昇は +6 パーセントポイント($p < 0.01$)です。境界では、`rstan-to-pystan` や `compile-compcert` のようなタスクが、メモリヘッドルームを得ることで成功率を大幅に改善しました。

これが測定にどう影響するか

おおよそ Terminal-Bench 仕様の 3x までは、追加リソースはインフラの信頼性問題——一時的なリソーススパイク——を修正します。Terminal-Bench メンテナーが使うサンドボックスプロバイダは裏で暗黙的にこれを行っています。eval は易しくなることなく、より安定になります。

しかし 3x を超えると、追加リソースは以前解けなかった問題をエージェントが解くのを能動的に助け始めます——制限が eval の測るものを実際に変えうることを示しています。厳しい制限は非常に効率的な戦略を不用意に報い、寛大な制限はより寛容で、利用可能なすべてのリソースをよりよく活用できるエージェントを報います。

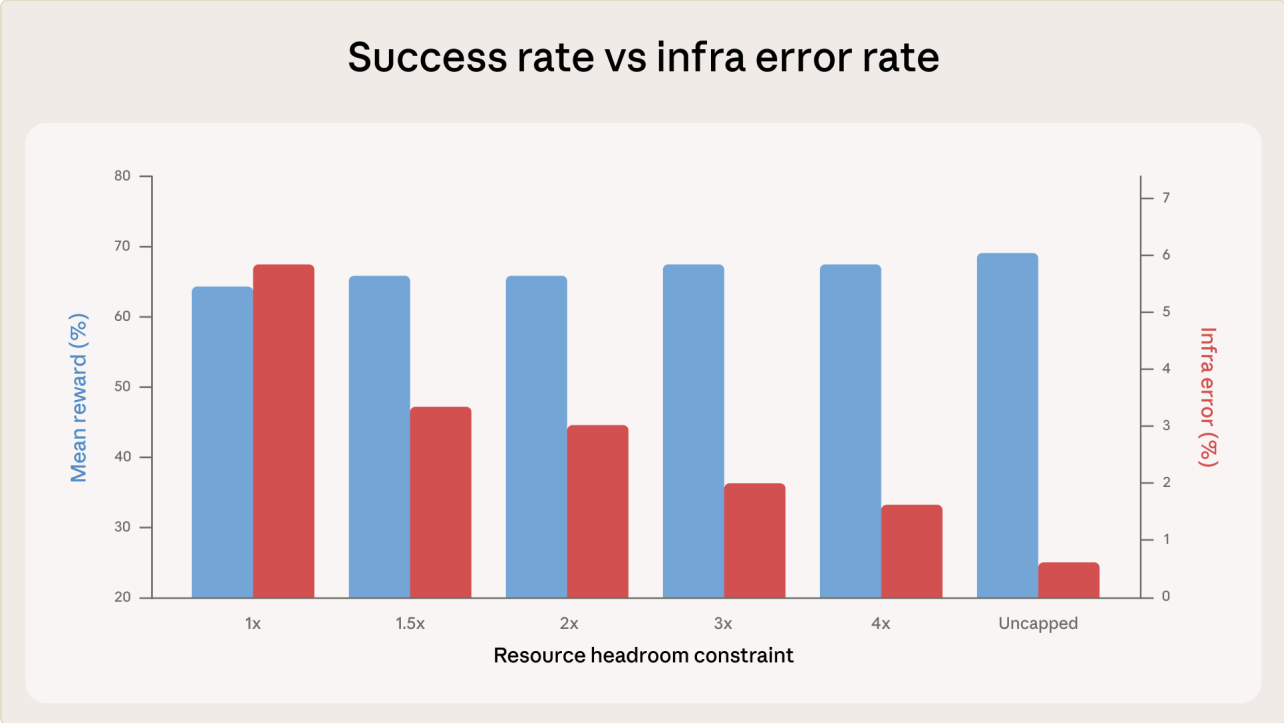
無駄なく効率的なコードを非常に速く書くエージェントは、厳しい制約下でよく機能します。重量級ツールで解をブルートフォースするエージェントは、寛大な制約下でよく機能します。どちらも正当にテストする価値がありますが、リソース構成を指定せずに 1 つのスコアに押し込めると、その違い——と現実世界への一般化可能

性——が解釈しづらくなります。

Bayesian ネットワークフィッティングを要求する Terminal-Bench タスク `bn-fit-modify` では、一部のモデルの最初の動きは標準的な Python データサイエンススタック（`pandas`、`networkx`、`scikit-learn`、およびそのツールチェーン全体）のインストールです。寛大な制限下ではこれが動きます。厳しい制限下では、エージェントが解のコードを 1 行も書く前に、インストール中に pod がメモリ不足になります。より無駄のない戦略（標準ライブラリだけで数学をスクラッチから実装する）は存在し、一部のモデルはそれをデフォルトとします。他は違います。異なるモデルは異なるデフォルトアプローチを持ち、リソース構成がそれらのアプローチのうちどれがたまたま成功するかを決めます。

私たちは中核的な発見を Anthropic の異なるモデルで再現しました。効果の方向は一貫しており、大きさは変動しました。同じ傾向は Claude 以外のモデルでも成り立つように見えますが、厳密にはテストしていません。

このパターンが Terminal-Bench 以外の eval でも成り立つかをテストするため、SWE-bench で交差実験も行いました。227 問題のベースラインの最大 5 倍まで RAM を変化させ、それぞれ 10 サンプルで。同じ効果が保たれましたが、大きさは小さかったです——スコアは再び RAM と共に単調に上がりましたが、5x は 1x よりわずか 1.54 パーセントポイント高いだけでした。SWE-bench タスクはリソース集約度が低いので小さな効果が予想されますが、リソース配分がそこでも中立ではないことを示しています。



分散の他のソース

リソース配分だけが隠し変数ではありません。ある構成では、時間制限も役割を演じ始めます。

原則として、クラスタの健康からハードウェア仕様、並行度から egress 帯域まで、evaluation セットアップのあらゆる要素が最終スコアに影響し得ます。エージェント型 eval は構造上エンドツーエンドのシステムテストであり、そのシステムのあらゆるコンポーネントが交絡因子として働きうるのです。私たちは逸話的に、合格率が時間帯と共に変動することを観察しました——おそらく API レイテンシがトラフィックパターンとインシデントで変わるからです。この効果を形式的に定量化してはいませんが、より大きな点を示しています——「モデル能力」と「インフラ挙動」の境界は、単一のベンチマークスコアが示唆するよりぼやけているということです。モデルプロバイダは eval インフラにハードウェアを専用化することでこれを遮蔽できますが、外部評価者は同じことをするのは難しいです。

公開ベンチマークは通常、純粋なモデル能力を測ることを意図していますが、実践的にはインフラのクセと混同するリスクがあります。場合によってはこれが望ましいこともあります——スタック全体のエンドツーエンドテストを可能にするからです——しかし多くの場合そうではありません。公開共有されるべきコーディング eval については、複数の時間と複数の日に実行してノイズを平均化するのが助けになります。

推奨事項

理想的なシナリオは、各 eval を同じ正確なハードウェア条件——eval を動かすスキャフォールドと推論スタックの両方——で動かすことです。これで完璧な再現性が全面に保証されます。しかし、これは常に実用的とは限りません。

コンテナランタイムが実際にリソースを強制する方法（保証された配分と別々の厳格 kill しきい値）を考えると、eval はタスクごとに両方のパラメータを指定すべきで、単一の固定値ではないことを勧めます。単一の正確な仕様は保証配分を kill しきい値と等しくし、マージンをゼロにします——1x で文書化した一時的なメモリスパイクは、eval を不安定にするのに十分です。2 つのパラメータを分離することで、スコア膨張を防ぐ厳格な上限を強制しつつ、コンテナに急な OOM kill を避けるだけの呼吸余地を与えられます。

両者の間の帯域は、床と天井のスコアが互いのノイズ内に収まるよう校正されるべきです。たとえば Terminal-Bench 2.0 で、タスクごとの仕様の 3x 天井はインフラエラー率を約 3 分の 2 削減（5.8% から 2.1%、$p < 0.001$）しつつ、スコア上昇を控えめで十分ノイズ内に保ちました（$p = 0.40$）。これは妥当なトレードオフです——インフラの交絡因子は主に中和され、意味のあるリソース圧力は取り除かれません。正確な倍率はベンチマークとタスク分布によって変わるので報告すべきですが、経験的校正原則は一般的です。

なぜこれを気にするか

これらの発見には eval インフラを超える実用的な帰結があります。ベンチマークスコアは意思決定入力としてますます使われていますが、この注目の増加（と依存）は必ずしも実行・報告方法の厳密さと歩調を合わせていません。現状では、リーダーボード上の 2 ポイントのリードは本物の能力差を反映しているかもしれないし、

一方の eval がより強力なハードウェアで、あるいはより幸運な時間帯で、あるいはその両方で走ったことを反映しているかもしれません。公開（あるいは標準化）されたセットアップ構成がなければ、関係者が同一条件下で客観的結果を再現する手間をかけない限り、外からは見分けるのが難しいです。

Anthropic のようなラボにとっての含意は、エージェント型 eval のリソース構成をプロンプト形式やサンプリング温度と同じ厳密さで文書化・制御される、第 1 級の実験変数として扱うべきだということです。ベンチマークメンテナーにとっては、推奨リソース仕様を公開すること（Terminal-Bench 2.0 がしているように）が大きく助けになり、強制手法を指定することで私たちが特定したギャップが埋まります。ベンチマーク結果を消費する誰にとっても、中核的なテイクアウェイは、エージェント型 eval 上の小さなスコア差は、報告された数字の精度が示唆するよりも不確実性を抱えているということです——特にいくつかの交絡因子は単に制御するには難しすぎます。

リソース手法が標準化されるまで、私たちのデータは、リーダーボードで 3 パーセントポイント未満の差は、eval 構成が文書化され一致するまで懐疑に値することを示唆しています。Terminal-Bench の中庸なリソース構成範囲にわたって観察される拡散は 2 パーセントポイントを少し下回ります。素朴な二項信頼区間は既に 1〜2 パーセントポイントにまたがります——ここで文書化したインフラの交絡因子はその上に積み重なるので、その内側ではありません。配分範囲の極端では、拡散は 6 に達します。

数ポイントのリードは本物の能力差を示しているかもしれない——あるいは単に VM がより大きいことを示しているだけかもしれません。

謝辞

執筆は Gian Segato。貢献への特別な感謝を Nicholas Carlini、Jeremy Hadfield、Mike Merrill、Alex Shaw に。本作業は、コーディングエージェントの評価に取り組む複数チームの集合的努力を反映しています。貢献に関心のある候補者は anthropic.com/careers から応募してください。